SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**
Question Count: 5

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: _____

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective	25%	Proposes	Reasonable	Basic	Ineffective

Action		effective and feasible corrections	corrections	corrections	corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

Student Name:	YADDALA KUSHAL KUMAR
Registration Number:	192524068
Course:	Cloud Computing and Big Data Analytics
Course Code:	CSA15
Assessment:	ASSESSMENT TOOL 2: Error Analysis Task
Institution:	SIMATS ENGINEERING
Course Outcome:	CO5: Design big data processing systems using the Hadoop ecosystem including
Taxonomy:	Dave's Taxonomy: Articulation (Level 4)
Total Marks:	25

ERROR ANALYSIS TASK

Question 1

A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

Answer

Identification of Error

The technical issue is **Data Skewness** (specifically **Partition Skew**). The failure occurs because the default HashPartitioner or the chosen MapReduce key distribution results in a disproportionate amount of data being sent to a single reducer, while others remain idle. This leads to a "long tail" problem where the job fails due to a **Time-out** or **Out-of-Memory (OOM) error** on the overloaded reducer, even though the cluster has sufficient aggregate resources.

Cause Analysis

- **Key Distribution:** The input data contains a high frequency of a specific key (e.g., a null value or a common category). Since Hadoop uses `key.hashCode() % numReducers` by default, all records with the same key are routed to the same reducer.
- **Input Split Inconsistency:** If the input files are non-splittable (e.g., compressed with Gzip without a container format), a single mapper may process a massive chunk of data, leading to uneven intermediate outputs.
- **Resource Starvation:** The overloaded reducer exceeds its allocated heap size (`mapreduce.reduce.memory.mb`), causing the TaskTracker/NodeManager to kill the process.

Corrective Actions

- **Implement a Custom Partitioner:** Develop a partitioner that incorporates a "salt" (a random prefix or suffix) to the keys to distribute high-frequency keys across multiple reducers.
- **Combiner Optimization:** Use a **Combiner** (mini-reducer) at the Map phase to aggregate data locally before it reaches the shuffle phase, significantly reducing the volume of data transferred to the reducers.

- **Secondary Sorting / Salting:** For keys that are naturally skewed, append a random integer to the key during the map phase and strip it during the reduce phase to ensure uniform distribution.
- **Adjust Split Size:** Use `mapreduce.input.fileinputformat.split.maxsize` to ensure mappers are not overwhelmed, and ensure input files are stored in splittable formats like BZip2 or LZO with indexing.

Hadoop Integration Concepts

This scenario highlights the importance of the **Shuffle and Sort phase** in the Hadoop ecosystem. Efficient resource utilization in YARN depends on the balance between `Map` output and `Reduce` input. By addressing the skew, the job transitions from a sequential bottleneck to a truly parallel distributed process, adhering to the core Hadoop principle of data locality and balanced computation.

Question 2

An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

Answer

Analysis of HDFS Data Unavailability

Identification of Error

The primary technical issue is a **violation of the HDFS Fault Tolerance principle**, specifically a **Replication Factor (RF) of 1**. In a standard HDFS deployment, data is partitioned into blocks and distributed across DataNodes. If the replication factor is set to 1, each block exists on only one physical node. Consequently, the failure of a single DataNode results in the immediate loss of all unique blocks hosted on that node, leading to partial file corruption and data unavailability across the cluster.

Cause Analysis

The root cause stems from a misconfiguration in the `hdfs-site.xml` file or a failure to account for **Rack Awareness**:

- **Low Replication Factor:** Setting `dfs.replication` to 1 removes redundancy. While this saves storage space, it creates a **Single Point of Failure (SPOF)** for every block of data.
- **NameNode Metadata Mismatch:** When a DataNode fails, the NameNode detects the absence of heartbeats. However, if no replicas exist elsewhere, the NameNode cannot initiate the **Replication Monitor** process to re-replicate the lost data from an existing source, rendering the data permanently unavailable until the node is recovered.
- **Under-replicated Blocks:** If the cluster was already in a state of "under-replication" due to previous ignored warnings, a single node failure could be the "tipping point" for critical metadata or data blocks.

Corrective Action

To restore high availability and fault tolerance, the following steps must be implemented:

- **Increase Replication Factor:** Modify the `dfs.replication` property in `hdfs-site.xml` to the default value of **3** . This ensures that even if two nodes fail simultaneously, the data remains accessible.
- **Manual Replication Trigger:** For existing files, use the Hadoop FS shell to increase replication: `hadoop fs -setrep -w 3 -R /`
- **Implement Rack Awareness:** Configure `net.topology.script.file.name` to ensure the NameNode places replicas across different racks, protecting against top-of-rack switch failures.
- **NameNode High Availability (HA):** Deploy a **Standby NameNode** using Quorum Journal Nodes (QJN) to ensure that the management of block metadata itself is not a bottleneck or a source of unavailability during node transitions.

Integration of Hadoop Concepts

In HDFS architecture, the **NameNode** manages the namespace and regulates access to files, while **DataNodes** manage storage. The system relies on the **Heartbeat mechanism** (every 3 seconds) and **Block Reports** to maintain cluster health. By maintaining a replication factor ≥ 2, HDFS leverages its self-healing nature where the NameNode automatically instructs DataNodes to copy blocks to new nodes upon detecting a failure, maintaining the desired state of reliability.

Question 3

A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

Answer

Identification of Error

The primary technical issue is a **lack of idempotency** within the data pipeline, where the retry mechanisms or overlapping ingestion schedules lead to the processing of the same data packets multiple times without a deduplication check at the sink.

Cause Analysis

- **NiFi Processor Overlap:** If the `ListFile` or `ListSFTP` processors are not configured with a persistent state, or if the `Fetch` processors fail before committing the state, the same files may be re-listed and re-ingested.
- **At-Least-Once Delivery Semantics:** Apache NiFi guarantees at-least-once delivery. If a network timeout occurs between NiFi and the analytics store (e.g., Hive or HBase) after the data is written but before the acknowledgment is received, NiFi will retry the flow, creating a duplicate.
- **Lack of Primary Key Constraints:** Many distributed analytics stores (like HDFS-based Hive tables) do not enforce unique constraints at the schema level, allowing identical records to be

appended during concurrent write operations.

Corrective Action

- **Implement Record-Level Deduplication:** Use the `DetectDuplicate` processor in NiFi, leveraging a distributed cache service (like Redis or Hazelcast) to store and check flowfile attributes (e.g., `hash` or `uuid`) before the final write.
- **Upsert Logic:** Configure the ETL sink to use **Upsert** (Update/Insert) operations rather than simple Appends. In Hive, this involves using ACID transactions; in HBase, this is handled naturally by row-key versioning.
- **Deterministic Naming:** Ensure source files are moved to a 'processed' directory immediately after ingestion to prevent re-scanning by the `List` processors.

Hadoop / Integration Concepts

In a Hadoop ecosystem, data integrity is maintained by ensuring that the **NiFi Provenance** data is aligned with the **HDFS metadata**. Utilizing **Apache Atlas** for lineage tracking can help identify where the duplication originated, while implementing a **Lambda Architecture** can allow for a batch-layer reconciliation to prune duplicates introduced by the speed-layer (NiFi).

Question 4

A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

Answer

Identification of Error

The technical issue is **Resource Starvation** caused by the default **FIFO (First-In-First-Out) Scheduling** policy in the YARN ResourceManager. In this scenario, the first large job submitted consumes all available cluster resources (CPU and Memory containers), forcing subsequent jobs from other users to remain in a `PENDING` or `WAITING` state indefinitely until the initial job completes. This leads to poor cluster utilization and high latency for smaller, time-sensitive tasks.

Cause Analysis

The root cause lies in the lack of **multi-tenancy resource isolation** .

- **Monolithic Allocation:** The FIFO scheduler does not support preemption or resource limits per user/queue.
- **Head-of-Line Blocking:** A single massive MapReduce or Spark job can saturate the `NodeManagers`, leaving zero capacity for concurrent requests.
- **Lack of Elasticity:** Without defined minimum guarantees or maximum caps, there is no mechanism to rebalance resources dynamically when demand increases across different user groups.

Corrective Action

To resolve resource starvation, the cluster should transition to a more sophisticated pluggable scheduler:

- **Capacity Scheduler:** Divide the cluster into hierarchical **Queues** (e.g., `Production`, `Development`). Assign each queue a guaranteed percentage of cluster resources. This ensures that even if one queue is overloaded, others have a reserved minimum capacity to execute jobs.
- **Fair Scheduler:** Aim to assign resources so that all running jobs get an equal share of resources over time. It supports **Preemption** , allowing the ResourceManager to kill containers from a job exceeding its fair share to satisfy a starving job.
- **Resource Limits:** Implement `max-applications` and `max-resource-capacity` limits per user to prevent a single user from overwhelming the scheduler's submission queue.

Hadoop Integration Concepts

In YARN (Yet Another Resource Negotiator), the **ResourceManager (RM)** uses the **Scheduler** component to allocate resources based on the `ResourceRequest` from the **ApplicationMaster (AM)**. By configuring `yarn-site.xml` to use the `CapacitySchedulerConfiguration`, administrators can enforce **Elasticity**, where idle resources from one queue can be temporarily used by another, but reclaimed (via preemption) when the rightful owner submits a job, ensuring both high utilization and fairness.

Question 5

A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Answer

Identification of Error

The root technical issue is a **Consistency-Availability-Partition (CAP) theorem trade-off violation**, specifically a failure in **Write-Read Consistency** (or Monotonic Read Consistency). The system is suffering from a **stale read** phenomenon where the recovery process retrieves a version of the data that lacks the most recent acknowledged writes, indicating that the backup was taken from a non-authoritative or lagging replica.

Cause Analysis

The design flaw likely stems from one of the following technical root causes:

- **Asynchronous Replication Lag:** If the backup process targets a secondary node in an asynchronous replication setup, the backup may capture a state before the latest transactions have propagated from the primary node.
- **Lack of Fencing/Quorum Validation:** During recovery, the system may fail to perform a **Quorum Read** ($R + W > N$). If the backup restoration does not validate the high-water mark or the Log Sequence Number (LSN) against the majority of the cluster, it accepts an outdated state as the 'truth'.

- **Split-Brain Recovery:** In a partitioned network, the backup might have been initiated from a minority partition that continued to operate without realizing it was disconnected from the latest updates.

Corrective Action

To resolve this flaw, the following architectural changes are required:

- **Synchronous Checkpointing:** Implement synchronous replication for metadata and critical state updates to ensure the backup source is always consistent with the last acknowledged write.
- **Version Vectoring:** Utilize **Vector Clocks** or **Generation Clock** stamps on all data blocks. During restoration, the system must compare the backup version against the cluster's metadata store to identify and reject stale epochs.
- **Read-Repair on Recovery:** Post-restoration, trigger a mandatory read-repair or anti-entropy protocol (like Hadoop's `fsck` or Cassandra's Hinted Handoff) to synchronize the restored data with any surviving replicas that may hold newer versions.

Integration with Hadoop Concepts

In a **Hadoop Distributed File System (HDFS)** context, this flaw relates to the **Edit Log** and **FsImage** synchronization. If a Secondary NameNode or Checkpoint Node performs a checkpoint while the Primary NameNode has uncommitted edits in memory that haven't been flushed to the journal (e.g., in a High Availability setup using Quorum Journal Manager), the resulting backup will be outdated. Ensuring the `dfs.namenode.edits.dir` is consistent across the JournalNodes is critical to preventing this recovery error.
